Pesquisa Reprodutível com R

Davi Athaydes Leite

11º Seminário Discente do IESP-UERJ

2026-10-28

Roteiro do curso

Vamos acompanhar um exemplo do começo ao fim: o **comparecimento eleitoral** nos municípios brasileiros, com dados do TSE. O percurso tem três etapas:

1. **Coleta dos dados**
   - onde estão os dados do TSE e como obtê-los?
   - como automatizar o download para vários anos de uma vez?
2. **Tratamento e organização**
   - como transformar dezenas de arquivos brutos em um único banco?
   - como usar o DuckDB para trabalhar com grandes volumes de dados?
3. **Elaboração de resultados**
   - como construir indicadores de participação eleitoral?
   - como apresentá-los em tabelas e gráficos?

Em cada etapa, alternamos entre os slides e os scripts no R.

Parte 1: Coleta dos dados

Onde estão os dados?

O TSE disponibiliza os resultados das eleições no **Portal de Dados Abertos** (dadosabertos.tse.jus.br).

Para o comparecimento, usamos o conjunto **detalhe da votação por município e zona**: aptos, comparecimento, abstenções, brancos e nulos.

Cada ano é um arquivo `.zip` em um endereço com o mesmo padrão:

```
cdn.tse.jus.br/estatistica/sead/odsele/
  detalhe_votacao_munzona/detalhe_votacao_munzona_2022.zip
```

Repare...

Só o **ano** muda de um endereço para outro. Isso é tudo de que precisamos para automatizar.

Automatizando o download

A função `download_tse()` repete, para cada ano:

1. monta o endereço do arquivo;
2. verifica se o `.csv` já existe na pasta e, se existir, **pula** o ano;
3. baixa o `.zip` e extrai apenas o arquivo nacional (`_BRASIL.csv`).

```
download_tse(
  file_name  = "detalhe_votacao_munzona",
  years      = seq(1998, 2024, 2),
  unzip_path = "data/dados_brutos/mun_turnout"
)
```

Rodar de novo não baixa tudo outra vez: o script pode ser executado do início ao fim quantas vezes for preciso.

Prática 1

`scripts/download_tse.R`: a função de download

`scripts/mun_turnout.R`: a chamada da função para 1998–2024

Parte 2: Tratamento e organização

Do bruto ao unificado

Temos um `.csv` por ano. Para cada arquivo:

- **lemos** com a codificação correta (`Latin-1`);
- **selecionamos** só as colunas de interesse, com `any_of()`, já que nem todo ano tem as mesmas colunas;
- **agregamos** de zona para município, somando as quantidades (`QT_*`).

Depois, **empilhamos** todos os anos em um único banco e **padronizamos** os textos (sem acentos, em maiúsculas).

```
lista_df[[x]] <- data.table::fread(x, encoding = "Latin-1") |>
  select(any_of(colunas)) |>
  group_by(ANO_ELEICAO, NR_TURNO, SG_UF,
           CD_MUNICIPIO, DS_CARGO) |>
  summarise(across(contains("QT_"), ~ sum(.x, na.rm = TRUE)))
```

Por que DuckDB?

O banco unificado de quase 30 anos de eleições é grande. O DuckDB:

- guarda as tabelas em **um único arquivo**, sem precisar de servidor;
- deixa consultar com os mesmos verbos do `dplyr`;
- só traz os dados para a memória quando pedimos (`collect()`).

```
con <- dbConnect(duckdb(), dbdir = "data/tratados/TSE")
dbWriteTable(con, "mun_turnout", mun_turnout,
             overwrite = TRUE)
```

Na prática

O tratamento roda **uma vez**. As análises seguintes partem do arquivo `TSE`, e não dos `.csv` brutos.

Prática 2

`scripts/mun_turnout.R`: leitura, unificação e gravação no DuckDB

Parte 3: Elaboração de resultados

Do banco ao painel

1. **Filtramos dentro do DuckDB**: anos de eleição presidencial, eleições ordinárias, cargos de presidente e governador;
2. trazemos para a memória só o resultado (`collect()`);
3. criamos uma dummy: **houve 2º turno para governador** naquela UF e ano?
4. calculamos os indicadores, em % do eleitorado apto:

$$\text{comparecimento} = \frac{\text{QT_COMPARECIMENTO}}{\text{QT_APTOS}} \times 100$$

e da mesma forma para **abstenção**, **brancos** e **nulos**.

O resultado é um painel **município × turno × eleição**.

Prática 3

`scripts/participacao_eleitoral.R`: do DuckDB ao painel de comparecimento